

PANDUAN ARSITEKTUR & TEKNIS

Sistem Informasi Administrasi RT (Apprentice Skill Fit Test)

Disusun Oleh: Senior System Analyst

Ditujukan Untuk: Full Stack Developer Candidate

Tanggal: 24 Juli 2026

1. Ringkasan Proyek & Tujuan

Dokumen ini ditujukan sebagai pedoman arsitektur dan teknis (guideline) untuk pengerjaan **Skill Fit Test - Full Stack Programmer** BEON Intermedia. Aplikasi ini bertujuan untuk mendigitalisasi proses administrasi RT dalam mengelola data penghuni, status rumah, iuran bulanan (satpam dan kebersihan), serta pengeluaran operasional.

CRITICAL CONSTRAINTS (WAJIB DIPATUHI):

- TIDAK BOLEH menggunakan Docker dalam bentuk apapun.
- Backend (Laravel) dan Frontend (React) WAJIB dibuat secara terpisah (Separate Repositories/Folders).
- DBMS wajib menggunakan MySQL.
- Output harus menyertakan ERD, Repo Aplikasi, dan Panduan Instalasi yang sangat jelas.

2. Arsitektur Sistem (High-Level)

Sistem akan mengadopsi pola **Decoupled Architecture (Client-Server)**:

- **Backend (RESTful API):** Dikembangkan menggunakan framework **PHP Laravel**. Bertanggung jawab atas autentikasi, business logic, pemrosesan transaksi, dan penyediaan data JSON untuk frontend.
- **Frontend (Single Page Application):** Dikembangkan menggunakan library **React JS**. Bertanggung jawab atas UI/UX, state management, dan interaksi pengguna. Disarankan menggunakan Vite sebagai bundler.
- **Database: MySQL.**
- **Komunikasi:** Pertukaran data menggunakan HTTP/HTTPS berformat JSON. Endpoint API harus diamankan (misalnya menggunakan Laravel Sanctum untuk token-based authentication).

3. Desain Database (Panduan ERD)

Untuk mengakomodasi kebutuhan bisnis terkait histori penghuni, status rumah, dan fleksibilitas pembayaran (bulanan vs tahunan), berikut rancangan skema relasional yang direkomendasikan:

1. Tabel users (Administrator / RT)

Mengelola akses login ke dalam sistem.

- id (PK), name, email, password, timestamps

2. Tabel houses (Data Rumah)

Menyimpan total 20 rumah dalam perumahan beserta status huniannya.

- id (PK)
- nomor_rumah (String, Unique)
- status_hunian (Enum: 'dihuni', 'tidak dihuni')

3. Tabel residents (Data Penghuni)

Master data untuk orang yang tinggal (atau pernah tinggal) di perumahan.

- id (PK)
- nama_lengkap (String)
- foto_ktp (String, path URL)
- status_penghuni (Enum: 'tetap', 'kontrak')
- nomor_telepon (String)
- status_pernikahan (Enum: 'menikah', 'belum menikah')

4. Tabel house_histories (Histori Rumah & Penghuni)

Memenuhi kriteria: *"Harus Terdapat catatan historical pada masing masing rumah siapa penghuninya"*.

- id (PK)
- house_id (FK to houses.id)
- resident_id (FK to residents.id)
- start_date (Date) - Kapan mulai menghuni
- end_date (Date, Nullable) - Kapan selesai menghuni. (Jika null berarti masih menghuni)

5. Tabel invoices (Tagihan Iuran Bulanan)

Digenerate setiap bulan untuk rumah yang "dihuni" dan seluruh penghuni "tetap".

- id (PK)
- house_id (FK to houses.id)
- resident_id (FK to residents.id)
- bulan (Integer 1-12)
- tahun (Integer)
- tagihan_kebersihan (Decimal/Int - Default: 15.000)
- tagihan_satpam (Decimal/Int - Default: 100.000)
- status_kebersihan (Enum: 'lunas', 'belum_lunas')
- status_satpam (Enum: 'lunas', 'belum_lunas')

6. Tabel transactions (Keuangan: Pemasukan & Pengeluaran)

Mempermudah pembuatan report (grafik 1 tahun) dan detail pengeluaran per bulan.

- id (PK)
- tipe (Enum: 'pemasukan', 'pengeluaran')
- kategori (Enum: 'iuran_kebersihan', 'iuran_satpam', 'perbaikan_jalan', 'gaji_satpam', dll)
- nominal (Decimal/Int)
- tanggal (Date)
- deskripsi (Text, Nullable)
- invoice_id (FK to invoices.id, Nullable - terisi jika tipe=pemasukan dari iuran)

4. Panduan Implementasi Fitur (Business Logic)

A. Pengelolaan Penghuni & Rumah

- **Foto KTP:** Pastikan Laravel menyimpan file KTP menggunakan Storage facade, symlink `storage:link` harus dijalankan, dan endpoint API mengembalikan Full URL gambar.
- **Assign Penghuni ke Rumah:** Saat Admin menempatkan penghuni A ke rumah B:
 1. Update tabel houses status menjadi "dihuni".
 2. Tutup record house_histories lama untuk rumah B (jika ada yang aktif) dengan set end_date.
 3. Buat record baru di house_histories dengan resident_id baru dan start_date hari ini.

B. Pengelolaan Pembayaran Iuran

- **Logic Tagihan:** Rumah dengan penghuni "tetap" selalu ditagih. 5 rumah lainnya HANYA ditagih jika tabel houses berstatus "dihuni".
- **Pembayaran Multi-Bulan (Tahunan):** Untuk mengakomodir kriteria *"penghuni yang membayar iuran bulanan 1 tahun, namun untuk iuran satpam bulanan"*:
 - Di Frontend, buat form pembayaran yang bisa mencentang beberapa bulan sekaligus untuk satu jenis iuran (misal: Bayar Kebersihan untuk Jan-Des = 12 bulan).
 - Backend menerima array [{bulan: 1, tahun: 2024, jenis: 'kebersihan'}, ...] lalu meng-update status_kebersihan = 'lunas' pada `invoices` terkait dan meng-insert ke tabel transactions.

C. Report & Dashboard (Grafik)

- **Summary Per Bulan:** Buat 1 endpoint khusus GET /api/reports/summary?year=2024. Backend melakukan agregasi `SUM(nominal)` dari tabel transactions di-group berdasarkan bulan dan tipe (pemasukan vs pengeluaran). Kalkulasi saldo sisa (Pemasukan - Pengeluaran) per bulan.
- Di Frontend, gunakan library seperti **Chart.js** atau **Recharts** untuk merender grafik bar/line selama 12 bulan.

5. Standar Kode & Repositori

- **Penamaan Folder Repo:** rt-admin-backend (Laravel) dan rt-admin-frontend (React).
- **Response API Standar:** Gunakan JSON wrapper yang konsisten, misal: { "status": "success", "message": "...", "data": {...} }
- **Panduan Instalasi (SANGAT PENTING):** Mengingat Docker dilarang, instruksi README.md harus mendetail untuk environment lokal (XAMPP/Laragon). Harus mencakup:
 1. Cara clone repo.
 2. composer install & npm install.
 3. Setup .env dan kredensial database default.
 4. Perintah migrasi dan seeder: php artisan migrate --seed (WAJIB sertakan dummy data 20 rumah agar penguji mudah mengetes).
 5. Perintah start server: php artisan serve dan npm run dev.
 6. Default akun login Admin (Email & Password).

Pesan Tambahan untuk Developer:

Pastikan setiap UI memiliki screenshot per fitur (sebagai lampiran dokumen terpisah sesuai instruksi tes). Jangan meremehkan validasi form (seperti nomor telepon, nominal angka, file upload KTP). Semakin rapi kode (clean code) dan UI/UX yang diberikan, semakin besar peluang kelulusan.